EXPERIMENT – 1
Familiarization of environments in R.

R provides an environment for statistical computing, data analysis, graphical representation, and programming. When RStudio is opened, it provides a graphical interface through which we can write and execute R programs.
The major components we need to become familiar with are:

1. R Console
2. Script Editor / Source Pane
3. Environment Pane
4. History Pane
5. Files Pane
6. Plots Pane

1.1: R Console: The Console is where R commands are directly entered and executed. The Console is useful for:

1. Executing commands immediately
2. Testing small pieces of code
3. Performing calculations
4. Checking functions
5. Viewing output

For larger programs, however, you should generally use the Script Editor.

1.2: Script Editor (Source Pane): The Script Editor is where you write and save R programs.

Create a new R script using: File → New File → R Script
You can also use: Ctrl + Shift + N
Save the file as: <filename>.R
The .R extension indicates an R source-code file.
You can execute individual lines or selected code from the script.

Why use a script? - Unlike typing everything directly into the Console, your code can be: Saved, Edited, Reused, Shared

1.3: Environment Pane: A graphical interface in RStudio that displays the objects currently available in the active R environment, along with information about their names, values, and types.

When you execute:

x < - 1

R creates an object called x.
Now look at the Environment pane.

You should see something similar to:
Environment
x 10

The Environment pane therefore provides a convenient view of the objects currently available in your R session.

1.4: History Pane: The History pane records commands that you have previously executed.

For example, if you run:

```
x <- 10
y <- 20
x + y
mean(c(10,20,30))
```

these commands will appear in the History.

This is useful when you:

- Forget a command you previously used
- Want to reuse an earlier command
- Want to inspect your recent work

1.6: Files Pane: The Files pane allows you to browse files and directories from within RStudio.

For example, you might have:

```
STUR/
├── Experiment1.R
├── Experiment2.R
├── data.csv
└── output/
```

You can navigate through these files directly from RStudio.

This becomes particularly useful when your practicals start involving datasets.

1.7: Plots Pane: The Plots pane displays graphs generated using R.

For example, execute:

```
x <- c(10, 20, 30, 40, 50)
plot(x)
```

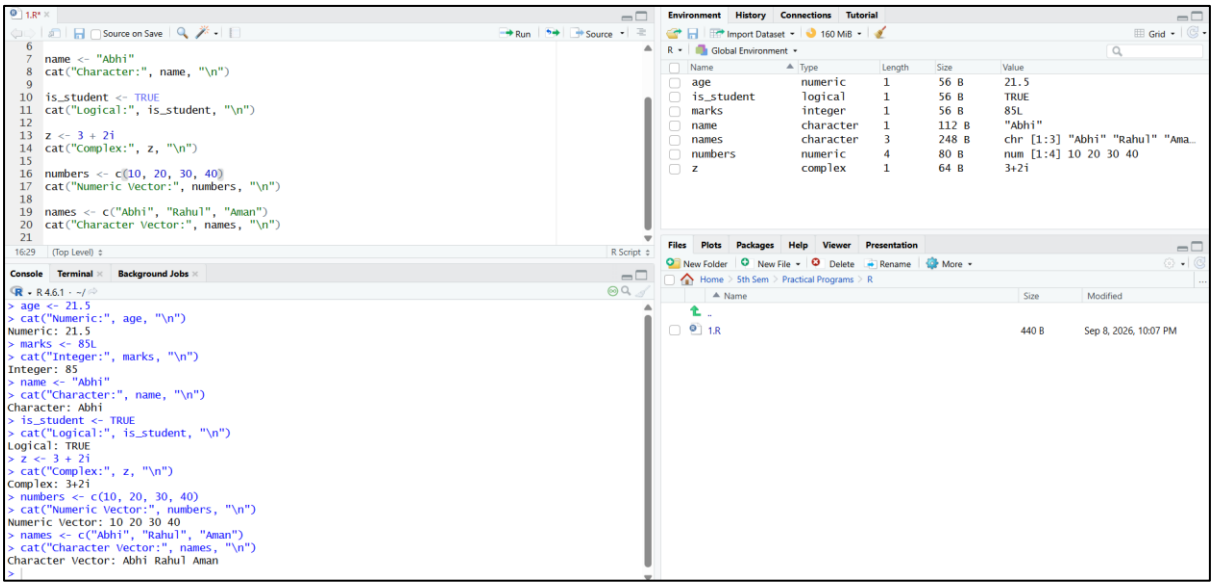
R will generate a graph, which should appear in the Plots pane.

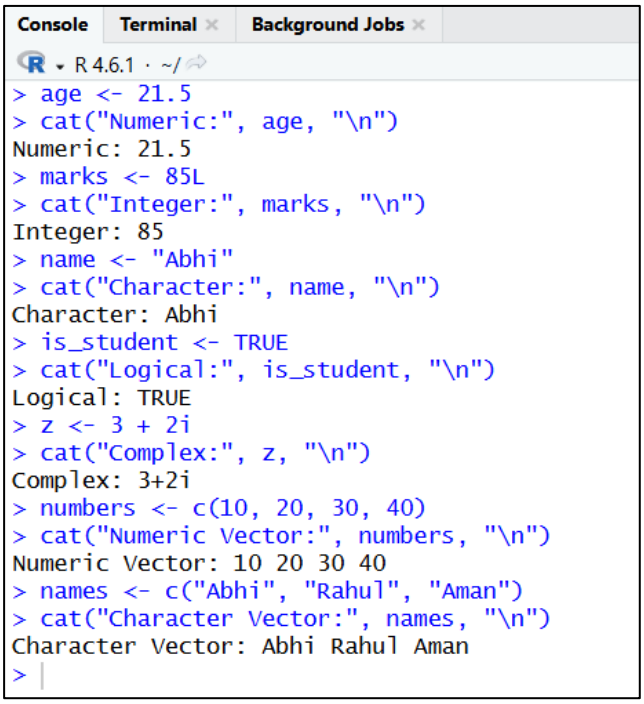
The Plots pane allows you to view and, depending on the plot, export the generated graph.

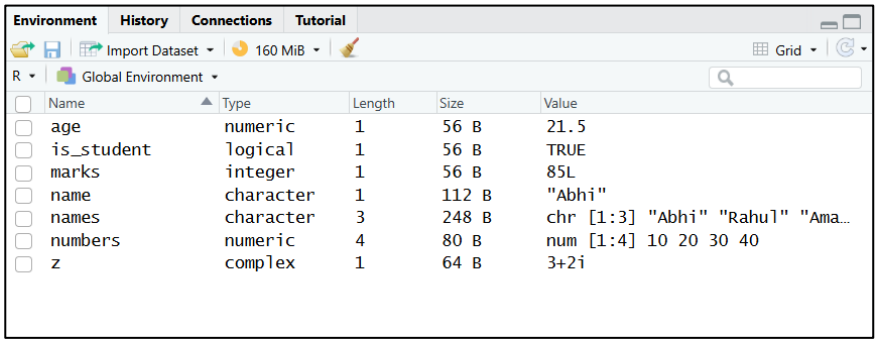
1.8: Sample Program

```
age <- 21.5
cat("Numeric:", age, "\n")
marks <- 85L
cat("Integer:", marks, "\n")
name <- "Abhi"
cat("Character:", name, "\n")
is_student <- TRUE
cat("Logical:", is_student, "\n")
z <- 3 + 2i
cat("Complex:", z, "\n")
numbers <- c(10, 20, 30, 40)
cat("Numeric Vector:", numbers, "\n")
names <- c("Abhi", "Rahul", "Aman")
cat("Character Vector:", names, "\n")
```

Output:


1.R*
Source on Save
Run
Source
6
7 name <- "Abhi"
8 cat("Character:", name, "\n")
9
10 is_student <- TRUE
11 cat("Logical:", is_student, "\n")
12
13 z <- 3 + 2i
14 cat("Complex:", z, "\n")
15
16 numbers <- c(10, 20, 30, 40)
17 cat("Numeric Vector:", numbers, "\n")
18
19 names <- c("Abhi", "Rahul", "Aman")
20 cat("Character Vector:", names, "\n")
21
16:29 (Top Level) R Script
Console Terminal Background Jobs
R 4.6.1 · ~/
> age <- 21.5
> cat("Numeric:", age, "\n")
Numeric: 21.5
> marks <- 85L
> cat("Integer:", marks, "\n")
Integer: 85
> name <- "Abhi"
> cat("Character:", name, "\n")
Character: Abhi
> is_student <- TRUE
> cat("Logical:", is_student, "\n")
Logical: TRUE
> z <- 3 + 2i
> cat("Complex:", z, "\n")
Complex: 3+2i
> numbers <- c(10, 20, 30, 40)
> cat("Numeric Vector:", numbers, "\n")
Numeric Vector: 10 20 30 40
> names <- c("Abhi", "Rahul", "Aman")
> cat("Character Vector:", names, "\n")
Character Vector: Abhi Rahul Aman
>
Environment History Connections Tutorial
Import Dataset 160 MiB
Grid
R Global Environment
Name Type Length Size Value
age numeric 1 56 B 21.5
is_student logical 1 56 B TRUE
marks integer 1 56 B 85L
name character 1 112 B "Abhi"
names character 3 248 B chr [1:3] "Abhi" "Rahul" "Ama...
numbers numeric 4 80 B num [1:4] 10 20 30 40
z complex 1 64 B 3+2i
Files Plots Packages Help Viewer Presentation
New Folder New File Delete Rename More
Home 5th Sem Practical Programs R
Name Size Modified
1.R 440 B Sep 8, 2026, 10:07 PM


Console Terminal Background Jobs
R 4.6.1 · ~/
> age <- 21.5
> cat("Numeric:", age, "\n")
Numeric: 21.5
> marks <- 85L
> cat("Integer:", marks, "\n")
Integer: 85
> name <- "Abhi"
> cat("Character:", name, "\n")
Character: Abhi
> is_student <- TRUE
> cat("Logical:", is_student, "\n")
Logical: TRUE
> z <- 3 + 2i
> cat("Complex:", z, "\n")
Complex: 3+2i
> numbers <- c(10, 20, 30, 40)
> cat("Numeric Vector:", numbers, "\n")
Numeric Vector: 10 20 30 40
> names <- c("Abhi", "Rahul", "Aman")
> cat("Character Vector:", names, "\n")
Character Vector: Abhi Rahul Aman
>


Environment History Connections Tutorial
Import Dataset 160 MiB
Grid
R Global Environment
Name Type Length Size Value
age numeric 1 56 B 21.5
is_student logical 1 56 B TRUE
marks integer 1 56 B 85L
name character 1 112 B "Abhi"
names character 3 248 B chr [1:3] "Abhi" "Rahul" "Ama...
numbers numeric 4 80 B num [1:4] 10 20 30 40
z complex 1 64 B 3+2i

EXPERIMENT – 2

Perform simple arithmetic's using R.

Code:

```
a <- 20
b <- 5

print("Addition:")
print(a + b)

print("Subtraction:")
print(a - b)

print("Multiplication:")
print(a * b)

print("Division:")
print(a / b)

print("Modulus:")
print(a %% b)

print("Integer Division:")
print(a %/% b)

print("Power:")
print(a ^ b)
```

Output:

```
> source("~/5th Sem/Practical Programs/R/2.R")
[1] "Addition:"
[1] 25
[1] "Subtraction:"
[1] 15
[1] "Multiplication:"
[1] 100
[1] "Division:"
[1] 4
[1] "Modulus:"
[1] 0
[1] "Integer Division:"
[1] 4
[1] "Power:"
[1] 3200000
```

EXPERIMENT – 3

Perform basic R functions.

Code:

```
numbers <- c(10, 20, 30, 40, 50)

# Basic mathematical functions
cat("Sum:", sum(numbers), "\n")
cat("Mean:", mean(numbers), "\n")
cat("Minimum:", min(numbers), "\n")
cat("Maximum:", max(numbers), "\n")
cat("Range:", range(numbers), "\n")
cat("Length:", length(numbers), "\n")

# Mathematical functions
cat("Absolute value:", abs(-25), "\n")
cat("Square root:", sqrt(25), "\n")
cat("Round:", round(12.5678, 2), "\n")
cat("Ceiling:", ceiling(12.3), "\n")
cat("Floor:", floor(12.8), "\n")

# Statistical functions
cat("Median:", median(numbers), "\n")
cat("Standard deviation:", sd(numbers), "\n")
cat("Variance:", var(numbers), "\n")

# Sorting functions
cat("Sorted values:", sort(numbers), "\n")
cat("Reverse sorted values:", sort(numbers, decreasing = TRUE), "\n")

# Sequence function
cat("Sequence:", seq(1, 10, by = 2), "\n")

# Repetition function
cat("Repeat:", rep(5, 4), "\n")

# Type checking functions
x <- 10
cat("Is numeric:", is.numeric(x), "\n")
cat("Is character:", is.character(x), "\n")
cat("Is integer:", is.integer(x), "\n")

# Conversion functions
y <- "100"
cat("Character to numeric:", as.numeric(y), "\n")

# Other useful functions
cat("Class:", class(numbers), "\n")
cat("Number of elements:", length(numbers), "\n")
```

Output:

```
> source("~/5th Sem/Practical Programs/R/3.R")
Sum: 150
Mean: 30
Minimum: 10
Maximum: 50
Range: 10 50
Length: 5
Absolute value: 25
Square root: 5
Round: 12.57
Ceiling: 13
Floor: 12
Median: 30
Standard deviation: 15.81139
Variance: 250
Sorted values: 10 20 30 40 50
Reverse sorted values: 50 40 30 20 10
Sequence: 1 3 5 7 9
Repeat: 5 5 5 5
Is numeric: TRUE
Is character: FALSE
Is integer: FALSE
Character to numeric: 100
Class: numeric
Number of elements: 5
```

EXPERIMENT – 4

Use various graphical techniques in EDA.

Code:

```
# Create sample data
students <- data.frame(
  Name = c("Abhi", "Saurav", "Mahira", "Shyam", "Shweta", "Akash", "Ibrahim", "Pritam"),
  Marks = c(85, 75, 90, 65, 80, 87, 23, 45),
  Hours = c(5, 3, 6, 2, 4, 3, 1, 2)
)
print(students)

# 1. Histogram
hist(
  students$Marks,
  main = "Histogram of Marks",
  xlab = "Marks"
)

# 2. Boxplot
boxplot(
  students$Marks,
  main = "Boxplot of Marks",
  ylab = "Marks"
)

# 3. Bar Plot
barplot(
  students$Marks,
  names.arg = students$Name,
  main = "Marks of Students",
  xlab = "Students",
  ylab = "Marks"
)

# 4. Pie Chart
pie(students$Marks,
    labels = students$Name,
    main = "Marks Distribution"
)

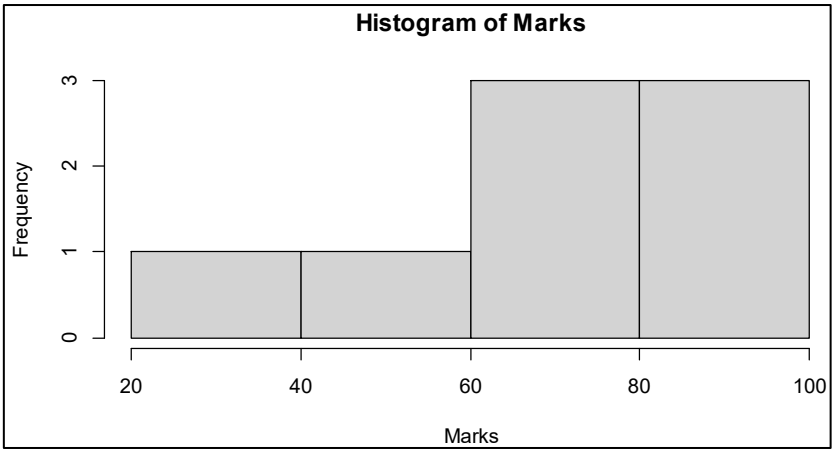
# 5. Scatter Plot
plot(
  students$Hours,
  students$Marks,
  main = "Study Hours vs Marks",
  xlab = "Study Hours",
  ylab = "Marks"
)
```

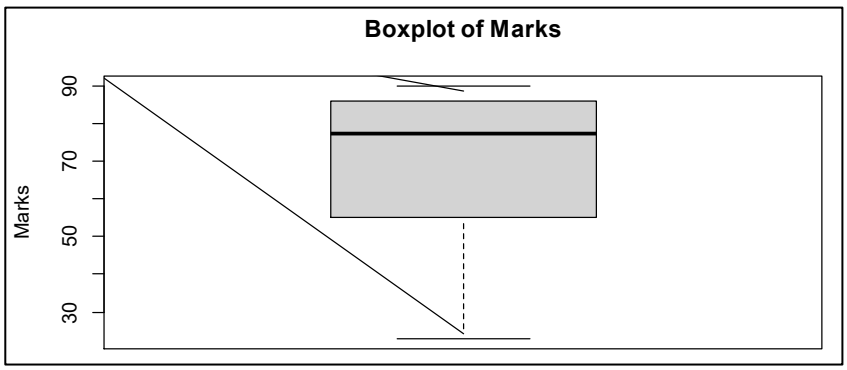
```
# 6. Line Plot
plot(
  students$Marks,
  type = "o",
  main = "Marks Trend",
  xlab = "Student",
  ylab = "Marks"
)
```

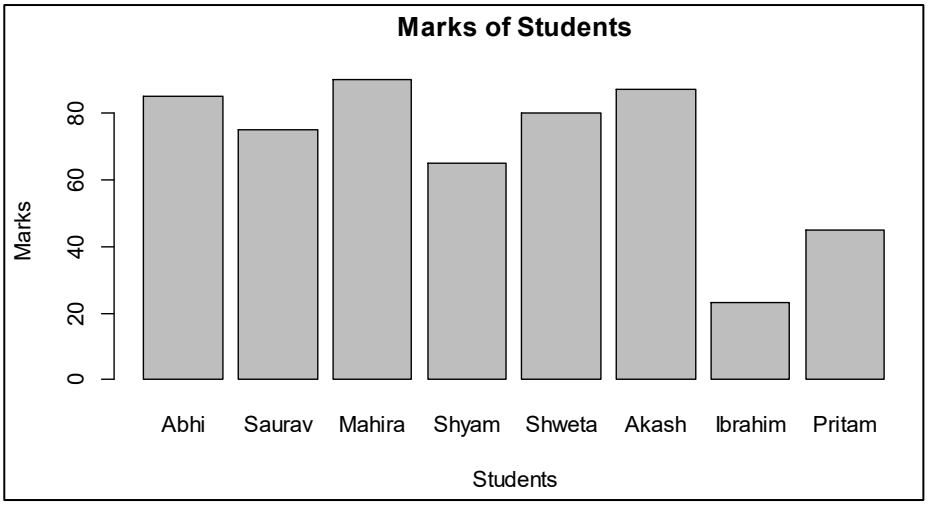
Output:

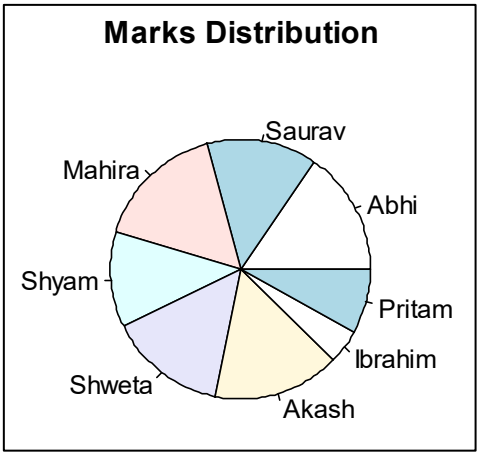
```
> source("~/5th Sem/Practical Programs/R/4.R")
      Name Marks Hours
1     Abhi    85     5
2   Saurav    75     3
3   Mahira    90     6
4    Shyam    65     2
5   Shweta    80     4
6    Akash    87     3
7 Ibrahim    23     1
8   Pritam    45     2
```

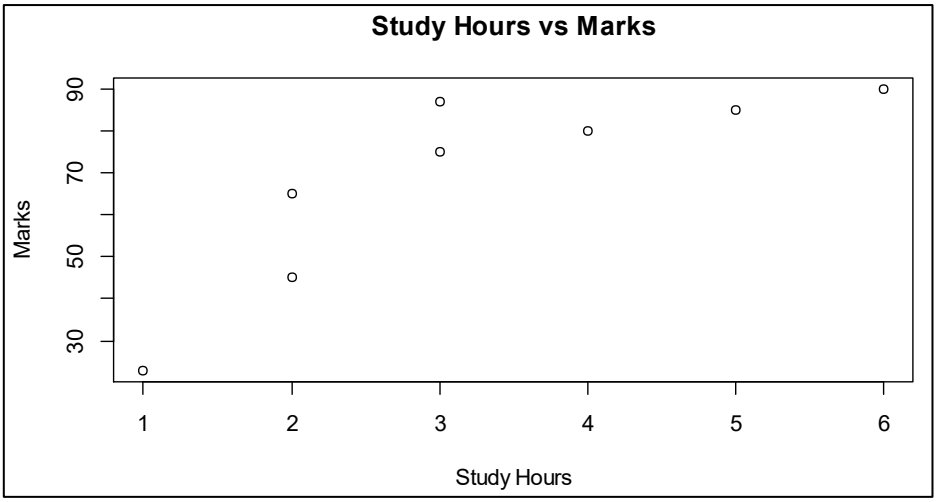
Plots Obtained:

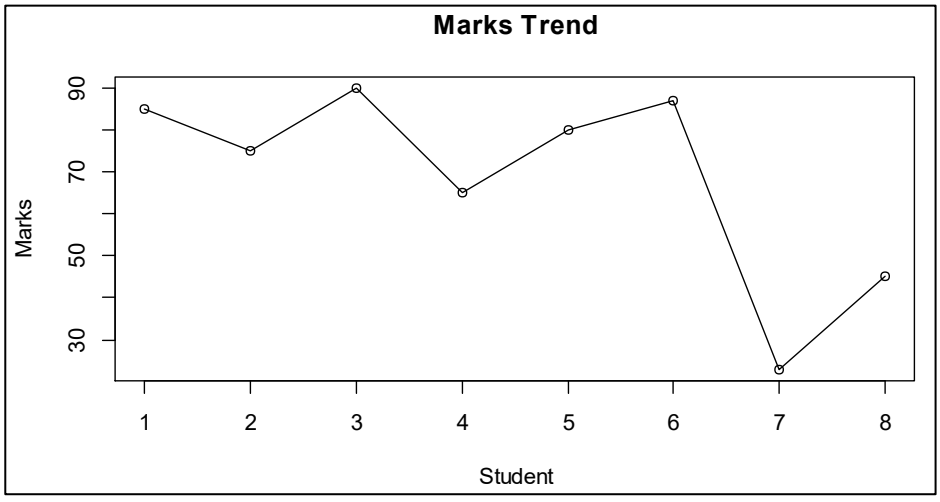





Marks of Students
Marks
0
20
40
60
80
Abhi
Saurav
Mahira
Shyam
Shweta
Akash
Ibrahim
Pritam
Students


Marks Distribution
Saurav
Mahira
Abhi
Shyam
Pritam
Ibrahim
Shweta
Akash


Study Hours vs Marks
Marks
30
50
70
90
1
2
3
4
5
6
Study Hours


Marks Trend
Marks
30
50
70
90
1
2
3
4
5
6
7
8
Student

EXPERIMENT – 5

Create different charts for visualization of given set of data.

Code:

```
# Load the built-in dataset
data(mtcars)

# Display the dataset
print(head(mtcars))

# 1. Bar Chart - Number of Cars by Cylinders
cylinder_count <- table(mtcars$cyl)

barplot(
  cylinder_count,
  names.arg = names(cylinder_count),
  main = "Cars by Number of Cylinders",
  xlab = "Number of Cylinders",
  ylab = "Number of Cars"
)

# 2. Histogram - Miles per Gallon
hist(
  mtcars$mpg,
  main = "Distribution of MPG",
  xlab = "Miles per Gallon",
  ylab = "Frequency"
)

# 3. Boxplot - MPG
boxplot(
  mtcars$mpg,
  main = "Boxplot of MPG",
  ylab = "Miles per Gallon"
)

# 4. Scatter Plot - MPG vs Weight
plot(
  mtcars$wt,
  mtcars$mpg,
  main = "MPG vs Weight",
  xlab = "Weight",
  ylab = "Miles per Gallon",
  pch = 19
)

# 5. Pie Chart - Transmission Type
pie(
  table(mtcars$am),
```

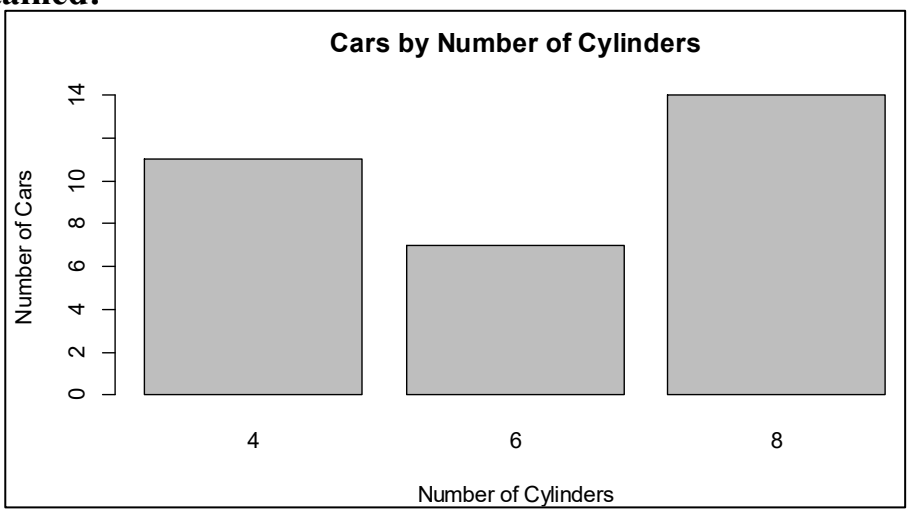
```
  labels = c("Automatic", "Manual"),
  main = "Transmission Types"
)

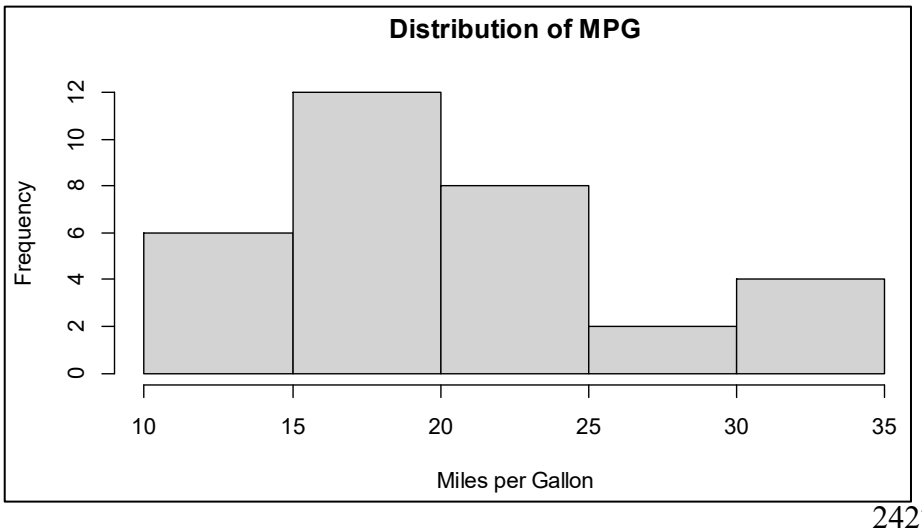
# 6. Line Chart - MPG of Cars
plot(
  mtcars$mpg,
  type = "o",
  main = "MPG of Cars",
  xlab = "Car Number",
  ylab = "Miles per Gallon"
)
```

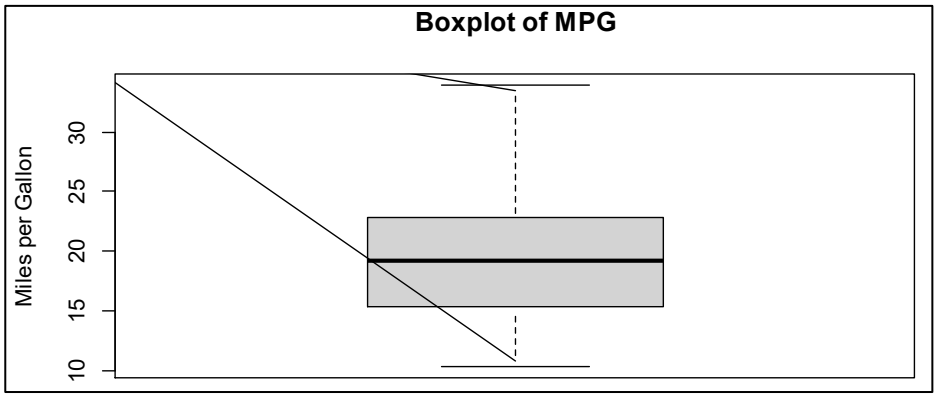
Output:

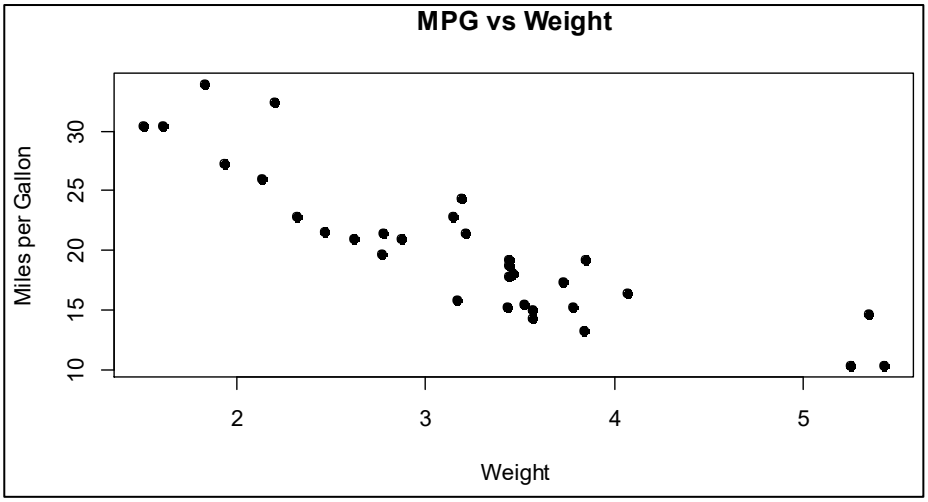
```
> source("~/5th Sem/Practical Programs/R/5.R")
                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
```

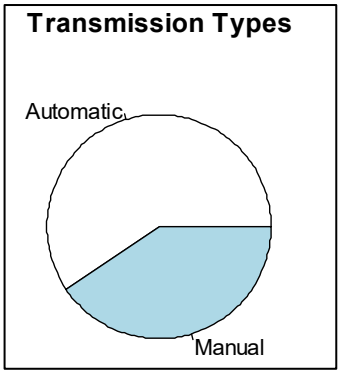
Plots Obtained:

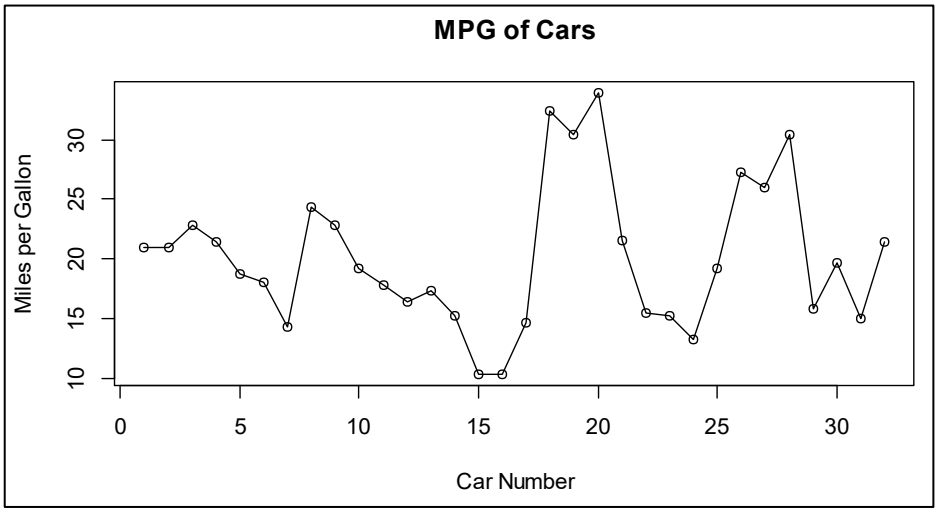





Boxplot of MPG
Miles per Gallon
10
15
20
25
30


MPG vs Weight
Miles per Gallon
10
15
20
25
30
2
3
4
5
Weight


Transmission Types
Automatic
Manual


MPG of Cars
Miles per Gallon
10
15
20
25
30
0
5
10
15
20
25
30
Car Number

EXPERIMENT – 6

Find the mean, median, standard deviation and quartiles of a set of observations.

Code:

```
x <- c(10, 80, 93, 56, 36, 6, 93, 72, 62, 43)

mean_value <- mean(x)
median_value <- median(x)
sd_value <- sd(x)
quartiles <- quantile(x)

cat("Given set of observations:", x, "\n\n")

cat("Number of Observations: ", length(x), "\n")
cat("Mean: ", mean(x), "\n")
cat("Median: ", median(x), "\n")
cat("Standard Deviation: ", sd(x), "\n")
cat("Quartiles:\n")
print(quartiles)
```

Output:

```
> source("~/5th Sem/Practical Programs/R/6.R")
Given set of observations: 10 80 93 56 36 6 93 72 62 43

Number of Observations:  10
Mean:  55.1
Median:  59
Standard Deviation:  31.23904
Quartiles:
   0%   25%   50%   75%  100%
 6.00 37.75 59.00 78.00 93.00
```

EXPERIMENT – 7

Find the Skewness and Kurtosis of a given dataset distribution.

Code:

```
library(moments)

data(iris)
cat("6 TOPMOST ROWS OF DATASET:\n")
print(head(iris))

skewness_values <- apply(iris[, 1:4], 2, skewness)
kurtosis_values <- apply(iris[, 1:4], 2, kurtosis)
cat("\nSKEWNESS VALUES OF ALL NUMERIC COLUMNS:\n")
print(skewness_values)
cat("\nKURTOSIS VALUES OF ALL NUMERIC COLUMNS:\n")
print(kurtosis_values)

hist(
  iris$Sepal.Length,
  main = "Distribution of Sepal Length with KDE",
  xlab = "Sepal Length",
  ylab = "Density",
  breaks = 20,
  border = "darkgreen",
  col = "lightgreen",
  ylim = c(0, 0.6),
  xlim = c(4, 8),
  freq = FALSE
)

kde_Sepal.Length <- density(iris$Sepal.Length)

lines(
  kde_Sepal.Length,
  col = "red",
  lwd = 2
)

hist(
  iris$Sepal.Width,
  main = "Distribution of Sepal Width with KDE",
  xlab = "Sepal Width",
  ylab = "Density",
  breaks = 20,
  border = "darkgreen",
  col = "lightgreen",
  ylim = c(0, 1.9),
  xlim = c(2, 4.5),
  freq = FALSE
```

```
)

kde_Sepal.Width <- density(iris$Sepal.Width)

lines(
  kde_Sepal.Width,
  col = "red",
  lwd = 2
)

hist(
  iris$Petal.Length,
  main = "Distribution of Petal Length with KDE",
  xlab = "Petal length",
  ylab = "Density",
  breaks = 20,
  border = "darkgreen",
  col = "lightgreen",
  ylim = c(0, 0.6),
  xlim = c(1, 6.9),
  freq = FALSE
)

kde_Petal.Length <- density(iris$Petal.Length)

lines(
  kde_Petal.Length,
  col = "red",
  lwd = 2
)

hist(
  iris$Petal.Width,
  main = "Distribution of Petal Width with KDE",
  xlab = "Petal Width",
  ylab = "Density",
  breaks = 20,
  border = "darkgreen",
  col = "lightgreen",
  ylim = c(0, 2.4),
  xlim = c(0.1, 2.6),
  freq = FALSE
)

kde_Petal_Width <- density(iris$Petal.Width)
lines(
  kde_Petal_Width,
  col = "red",
  lwd = 2
)
```

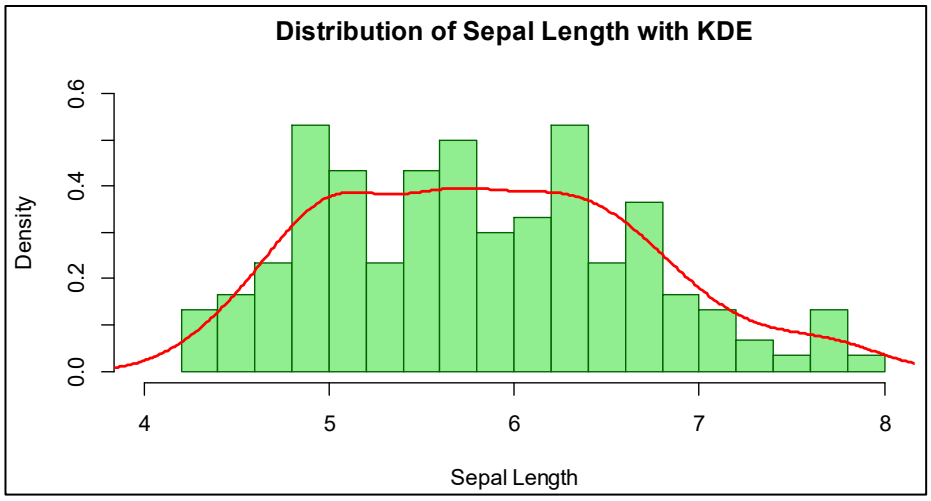
Output:

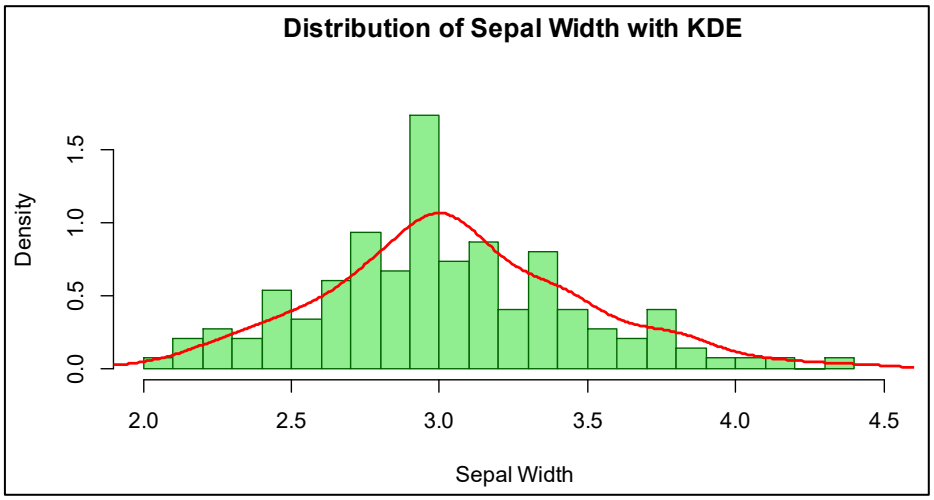
```
6 TOPMOST ROWS OF DATASET:
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
1          5.1         3.5          1.4         0.2  setosa
2          4.9         3.0          1.4         0.2  setosa
3          4.7         3.2          1.3         0.2  setosa
4          4.6         3.1          1.5         0.2  setosa
5          5.0         3.6          1.4         0.2  setosa
6          5.4         3.9          1.7         0.4  setosa

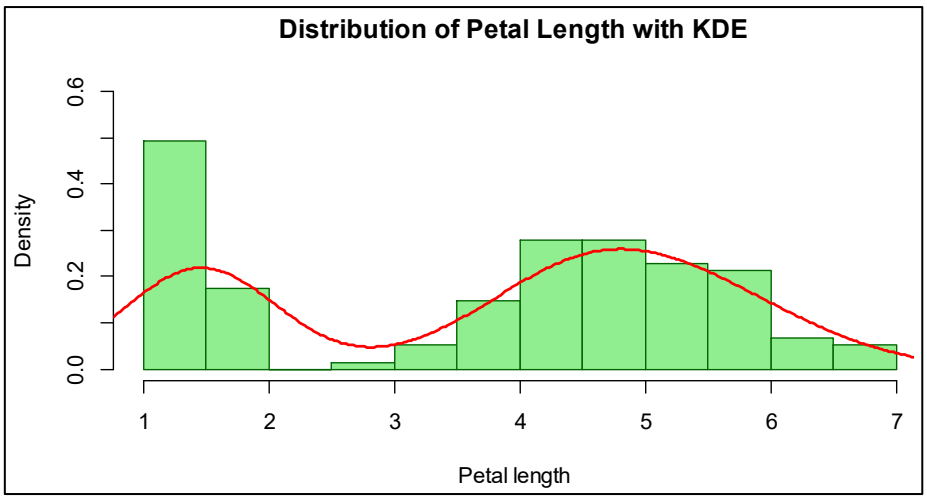
SKEWNESS VALUES OF ALL NUMERIC COLUMNS:
Sepal.Length  Sepal.Width Petal.Length  Petal.Width
   0.3117531    0.3157671   -0.2721277   -0.1019342

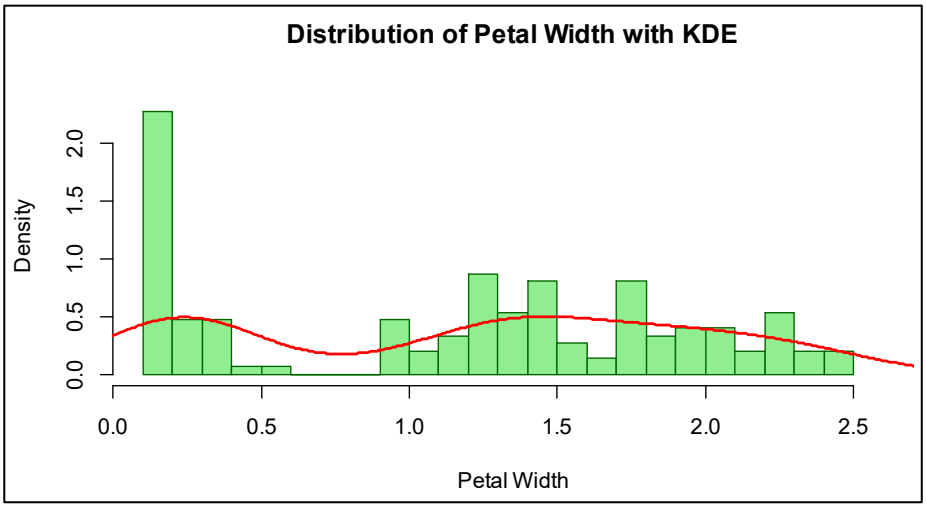
KURTOSIS VALUES OF ALL NUMERIC COLUMNS:
Sepal.Length  Sepal.Width Petal.Length  Petal.Width
    2.426432     3.180976     1.604464     1.663933
```

Plots Obtained:






Distribution of Petal Length with KDE
Density
0.0
0.2
0.4
0.6
1
2
3
4
5
6
7
Petal length


Distribution of Petal Width with KDE
Density
0.0
0.5
1.0
1.5
2.0
0.0
0.5
1.0
1.5
2.0
2.5
Petal Width

EXPERIMENT – 8

Given the scenario, implement the Bayes rule by finding the posterior probability.

Code:

```
data(mtcars)

# Define events:
# A = automatic transmission (am = 0)
# B = 8 cylinders (cyl = 8)

# Calculate prior probability P(A)
P_A <- mean(mtcars$am == 0)

# Calculate likelihood P(B | A)
P_B_given_A <- mean(mtcars$cyl[mtcars$am == 0] == 8)

# Calculate evidence P(B)
P_B <- mean(mtcars$cyl == 8)

# Apply Bayes' theorem
P_A_given_B <- (P_B_given_A * P_A) / P_B

# Display results
cat("Prior P(A):", P_A, "\n")
cat("Likelihood P(B|A):", P_B_given_A, "\n")
cat("Evidence P(B):", P_B, "\n")
cat("Posterior P(A|B):", P_A_given_B, "\n")
cat("Posterior percentage:", P_A_given_B * 100, "%\n")
```

Output:

```
Prior P(A): 0.59375
Likelihood P(B|A): 0.6315789
Evidence P(B): 0.4375
Posterior P(A|B): 0.8571429
Posterior percentage: 85.71429 %
```

EXPERIMENT – 9

Find the mass function of a binomial distribution with n=20,p=0.4n=20,p=0.4. Also draw the graphs of the mass function and cumulative distribution function.

Code:

```
# Parameters of the binomial distribution
n <- 20
p <- 0.4

# Possible numbers of successes
x <- 0:n

# Probability Mass Function (PMF)
pmf <- dbinom(x, size = n, prob = p)

# Cumulative Distribution Function (CDF)
cdf <- pbinom(x, size = n, prob = p)

# Display the probability table
result <- data.frame(
  x = x,
  PMF = pmf,
  CDF = cdf
)
print(result)

# Draw both graphs side by side
par(mfrow = c(1, 2))

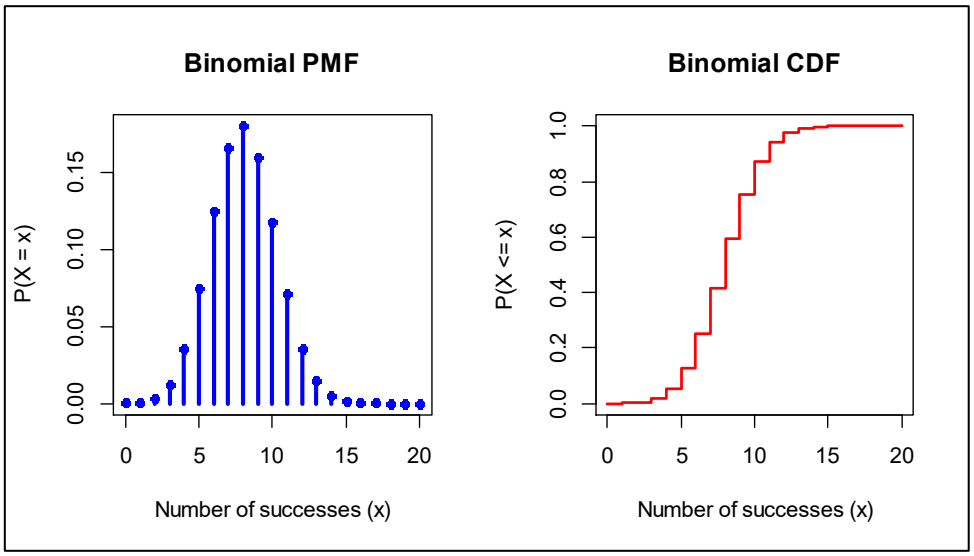
# Graph 1: PMF
plot(x, pmf,
     type = "h",
     lwd = 3,
     col = "blue",
     main = "Binomial PMF",
     xlab = "Number of successes (x)",
     ylab = "P(X = x)")
points(x, pmf, pch = 16, col = "blue")

# Graph 2: CDF
plot(x, cdf,
     type = "s",
     lwd = 2,
     col = "red",
     main = "Binomial CDF",
     xlab = "Number of successes (x)",
     ylab = "P(X <= x)",
     ylim = c(0, 1))
```

Output:

	x	PMF	CDF
1	0	3.656158e-05	3.656158e-05
2	1	4.874878e-04	5.240494e-04
3	2	3.087423e-03	3.611472e-03
4	3	1.234969e-02	1.596116e-02
5	4	3.499079e-02	5.095195e-02
6	5	7.464702e-02	1.255990e-01
7	6	1.244117e-01	2.500107e-01
8	7	1.658823e-01	4.158929e-01
9	8	1.797058e-01	5.955987e-01
10	9	1.597385e-01	7.553372e-01
11	10	1.171416e-01	8.724788e-01
12	11	7.099488e-02	9.434736e-01
13	12	3.549744e-02	9.789711e-01
14	13	1.456305e-02	9.935341e-01
15	14	4.854351e-03	9.983885e-01
16	15	1.294494e-03	9.996830e-01
17	16	2.696862e-04	9.999527e-01
18	17	4.230371e-05	9.999950e-01
19	18	4.700412e-06	9.999997e-01
20	19	3.298535e-07	1.000000e+00
21	20	1.099512e-08	1.000000e+00

Graphs Obtained:



EXPERIMENT – 10

Generate and draw the cdf and pdf of a normal distribution with mean=10 and standard deviation=3. Use values of x from 0 to 20 in intervals of 1.

Code:

```
# Generate x values from 0 to 20
x <- seq(0, 20, by = 1)

# Calculate PDF and CDF
pdf <- dnorm(x, mean = 10, sd = 3)
cdf <- pnorm(x, mean = 10, sd = 3)

result <- data.frame(x, PDF = pdf, CDF = cdf)
print(result)

par(mfrow = c(1, 2))

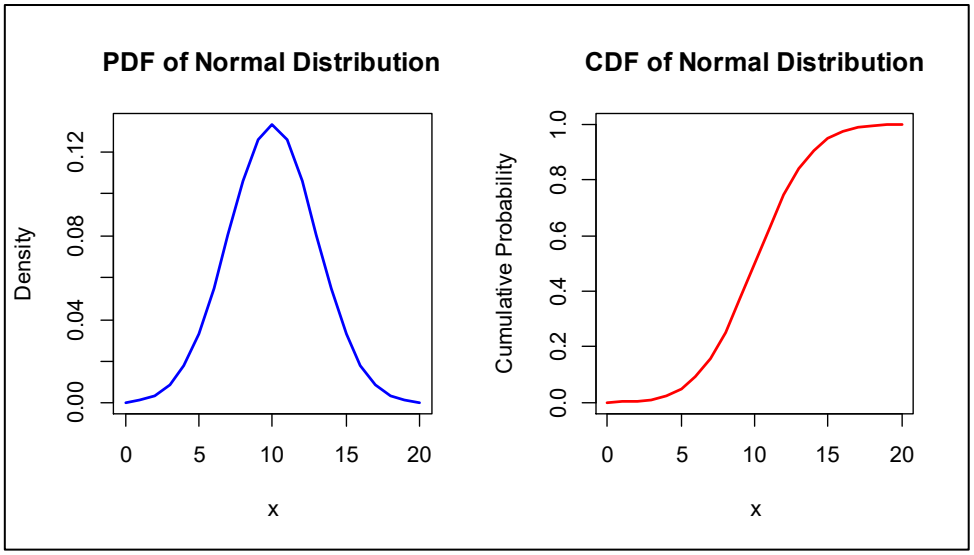
# Plot PDF
plot(x, pdf, type = "l",
     main = "PDF of Normal Distribution",
     xlab = "x", ylab = "Density",
     col = "blue", lwd = 2)

# Plot CDF
plot(x, cdf, type = "l",
     main = "CDF of Normal Distribution",
     xlab = "x", ylab = "Cumulative Probability",
     col = "red", lwd = 2)
```

Output:

	x	PDF	CDF
1	0	0.000514093	0.0004290603
2	1	0.001477283	0.0013498980
3	2	0.003798662	0.0038303806
4	3	0.008740630	0.0098153286
5	4	0.017996989	0.0227501319
6	5	0.033159046	0.0477903523
7	6	0.054670025	0.0912112197
8	7	0.080656908	0.1586552539
9	8	0.106482669	0.2524925375
10	9	0.125794409	0.3694413402
11	10	0.132980760	0.5000000000
12	11	0.125794409	0.6305586598
13	12	0.106482669	0.7475074625
14	13	0.080656908	0.8413447461
15	14	0.054670025	0.9087887803
16	15	0.033159046	0.9522096477
17	16	0.017996989	0.9772498681
18	17	0.008740630	0.9901846714
19	18	0.003798662	0.9961696194
20	19	0.001477283	0.9986501020
21	20	0.000514093	0.9995709397

Plots Obtained:



EXPERIMENT – 11

Construct a scatter plot to investigate the relationship between two variables.

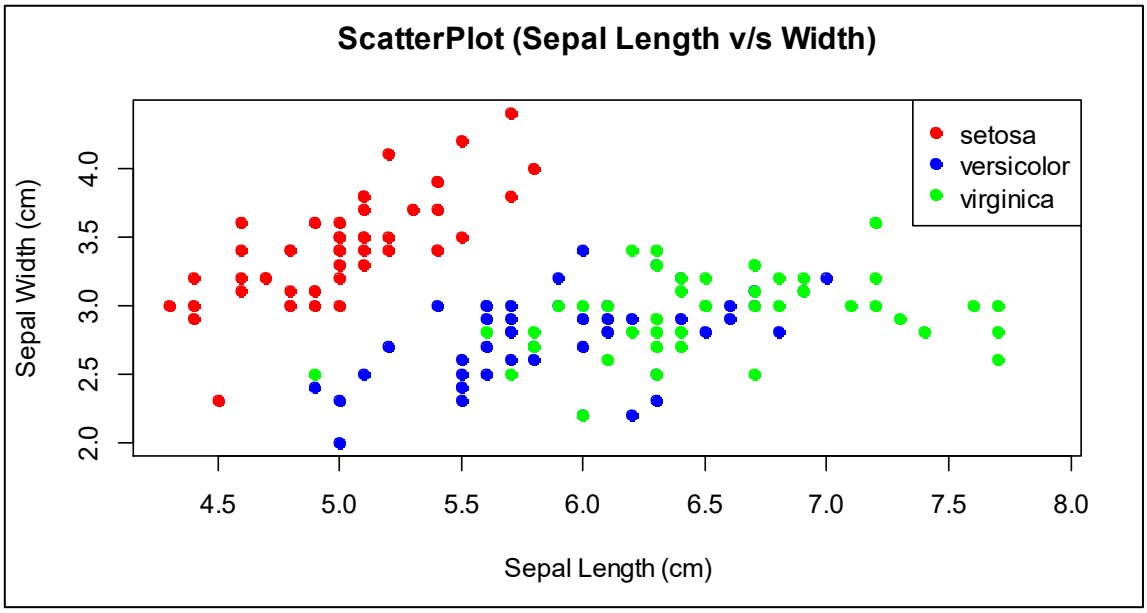
Code:

```
# Load the built-in dataset
data(iris)

# Create a scatter plot
plot(iris$Sepal.Length, iris$Sepal.Width,
     main = "ScatterPlot (Sepal Length v/s Width)",
     xlab = "Sepal Length (cm)",
     ylab = "Sepal Width (cm)",
     col = c("red", "blue", "green")[iris$Species],
     pch = 19)

# Add a legend
legend("topright",
       legend = levels(iris$Species),
       col = c("red", "blue", "green"),
       pch = 19)
```

Plot Obtained:



EXPERIMENT – 12

Perform the Z- test for single proportion, single mean etc.

12.1: Z-Test for Single Mean

Aim: To test whether the population mean of sepal length differs significantly from a specified value using the built-in iris dataset.

Hypotheses:

- $H_0: \mu = 5.8$
- $H_1: \mu \neq 5.8$

We will use the sample mean and sample size from iris, and assume a known population standard deviation of 0.8 cm for this exercise.

Code:

```
data(iris)

x <- iris$Sepal.Length
mu0 <- 5.8
sigma <- 0.8
n <- length(x)

# Calculate sample mean and Z-statistic
x_bar <- mean(x)
z <- (x_bar - mu0) / (sigma / sqrt(n))

# Calculate two-tailed p-value
p_value <- 2 * pnorm(-abs(z))

cat("Sample Mean =", x_bar, "\n")
cat("Z-statistic =", z, "\n")
cat("P-value =", p_value, "\n")

if (p_value < 0.05) {
  cat("Reject the null hypothesis")
} else {
  cat("Do not reject the null hypothesis")
}
```

Output:

```
Sample Mean = 5.843333
Z-statistic = 0.6634035
P-value = 0.5070722
Do not reject the null hypothesis
```

Conclusion: At the 5% significance level, there is insufficient evidence to conclude that the population mean sepal length differs from 5.8 cm, under the assumed population standard deviation.

12.2: Z-Test for Single Proportion

Aim: To test whether the proportion of a particular flower species in the iris dataset differs significantly from a specified population proportion. For this example, the success category is a flower belonging to the setosa species.

Hypotheses:

- $H_0: p = 0.30$
- $H_1: p \neq 5.8$

The hypothesized proportion is 30%.

Code:

```
data(iris)

n <- nrow(iris)
x <- sum(iris$Species == "setosa")
p0 <- 0.30

# Sample proportion
p_hat <- x / n

# Calculate Z-statistic
z <- (p_hat - p0) / sqrt(p0 * (1 - p0) / n)

# Calculate two-tailed p-value
p_value <- 2 * pnorm(-abs(z))

cat("Number of Setosa flowers =", x, "\n")
cat("Sample Proportion =", p_hat, "\n")
cat("Z-statistic =", z, "\n")
cat("P-value =", p_value, "\n")

if (p_value < 0.05) {
  cat("Reject the null hypothesis")
} else {
  cat("Do not reject the null hypothesis")
}
```

Output:

```
Number of Setosa flowers = 50
Sample Proportion = 0.3333333
Z-statistic = 0.8908708
P-value = 0.3729985
Do not reject the null hypothesis
```

Conclusion: At the 5% significance level, there is insufficient evidence that the population proportion differs from 30%.

12.3: Z-Test for Difference between Two Means

Aim: To test whether the mean sepal length differs significantly between two species using the built-in iris dataset. We compare setosa and versicolor flowers.

Hypotheses:

- $H_0: \mu_1 - \mu_2 = 0$
- $H_1: \mu_1 - \mu_2 \neq 0$

Assume the population standard deviation for each group is (0.8) cm.

Code:

```
data(iris)

x1 <- iris$Sepal.Length[iris$Species == "setosa"]
x2 <- iris$Sepal.Length[iris$Species == "versicolor"]

n1 <- length(x1)
n2 <- length(x2)
mu1 <- mean(x1)
mu2 <- mean(x2)
sigma1 <- 0.8
sigma2 <- 0.8

# Calculate Z-statistic
z <- (mu1 - mu2) / sqrt(sigma1^2/n1 + sigma2^2/n2)

# Two-tailed p-value
p_value <- 2 * pnorm(-abs(z))

cat("Mean of Setosa =", mu1, "\n")
cat("Mean of Versicolor =", mu2, "\n")
cat("Z-statistic =", z, "\n")
cat("P-value =", p_value, "\n")

if (p_value < 0.05) {
  cat("Reject the null hypothesis")
} else {
  cat("Do not reject the null hypothesis")
}
```

Output:

```
Mean of Setosa = 5.006
Mean of Versicolor = 5.936
Z-statistic = -5.8125
P-value = 6.154668e-09
Reject the null hypothesis
```

Conclusion: The mean sepal lengths of the two species differ significantly at the 5% significance level, assuming the specified population standard deviations.

12.4: Z-Test for Difference between two proportions

Aim: To test whether survival proportions differ significantly between males and females using the built-in Titanic dataset.Here, a success means that a passenger survived.

Hypotheses:

- $H_0: p_1 = p_2$
- $H_1: p_1 \neq p_2$

Code:

```
data(Titanic)

# Convert the table into a data frame
d <- as.data.frame(Titanic)

# Total passengers by sex
totals <- aggregate(Freq ~ Sex, data = d, sum)

# Number of survivors by sex
survivors <- aggregate(
  Freq ~ Sex,
  data = subset(d, Survived == "Yes"),
  sum
)

n1 <- totals$Freq[totals$Sex == "Female"]
n2 <- totals$Freq[totals$Sex == "Male"]

x1 <- survivors$Freq[survivors$Sex == "Female"]
x2 <- survivors$Freq[survivors$Sex == "Male"]

p1 <- x1 / n1
p2 <- x2 / n2

# Pooled proportion
p_pool <- (x1 + x2) / (n1 + n2)

# Calculate Z-statistic
se <- sqrt(p_pool * (1 - p_pool) * (1/n1 + 1/n2))

z <- (p1 - p2) / se

# Two-tailed p-value
p_value <- 2 * pnorm(-abs(z))

cat("Female Survival Proportion =", p1, "\n")
cat("Male Survival Proportion =", p2, "\n")
cat("Z-statistic =", z, "\n")
cat("P-value =", p_value, "\n")
```

```
if (p_value < 0.05) {
  cat("Reject the null hypothesis")
} else {
  cat("Do not reject the null hypothesis")
}
```

Output:

```
Female Survival Proportion = 0.7319149
Male Survival Proportion = 0.2120162
Z-statistic = 21.37461
P-value = 2.302151e-101
Reject the null hypothesis
```

Conclusion: The survival proportions of male and female passengers differ significantly at the 5% significance level.

EXPERIMENT – 13

Calculate the regression coefficient and obtain the lines of regression for the given data.

Code: